

Average Case Analysis of Java 7's Dual Pivot Quicksort^{*}

Sebastian Wild and Markus E. Nebel

Fachbereich Informatik, Technische Universität Kaiserslautern
`{s_wild,nebel}@cs.uni-kl.de`

Abstract. Recently, a new Quicksort variant due to Yaroslavskiy was chosen as standard sorting method for Oracle's Java 7 runtime library. The decision for the change was based on empirical studies showing that on average, the new algorithm is faster than the formerly used classic Quicksort. Surprisingly, the improvement was achieved by using a dual pivot approach, an idea that was considered not promising by several theoretical studies in the past. In this paper, we identify the reason for this unexpected success. Moreover, we present the first precise average case analysis of the new algorithm showing e.g. that a random permutation of length n is sorted using $1.9n \ln n - 2.46n + \mathcal{O}(\ln n)$ key comparisons and $0.6n \ln n + 0.08n + \mathcal{O}(\ln n)$ swaps.

1 Introduction

Due to its efficiency in the average, Quicksort has been used for decades as general purpose sorting method in many domains, e.g. in the C and Java standard libraries or as UNIX's system sort. Since its publication in the early 1960s by Hoare [1], classic Quicksort (Algorithm 1) has been intensively studied and many modifications were suggested to improve it even further, one of them being the following: Instead of partitioning the input file into two subfiles separated by a single pivot, we can create s partitions out of $s - 1$ pivots.

Sedgewick considered the case $s = 3$ in his PhD thesis [2]. He proposed and analyzed the implementation given in Algorithm 2. However, this dual pivot Quicksort variant turns out to be clearly inferior to the much simpler classic algorithm. Later, Hennequin studied the comparison costs for any constant s in his PhD thesis [3], but even for arbitrary $s \geq 3$, he found no improvements that would compensate for the much more complicated partitioning step [4]. These negative results may have discouraged further research along these lines.

Recently, however, Yaroslavskiy proposed the new dual pivot Quicksort implementation as given in Algorithm 3 at the Java core library mailing list [5]. He

^{*} This research was supported by DFG grant NE 1379/3-1.

¹ When s depends on n , we basically get the Samplesort algorithm from [4]. [5], [6] or [7] show that Samplesort can beat Quicksort if hardware features are exploited. [2] even shows that Samplesort is asymptotically optimal with respect to comparisons. Yet, due to its inherent intricacies, it has not been used much in practice.

² The discussion is archived at <http://permalink.gmane.org/gmane.comp.java.openjdk.core-libs.devel/2628>.

Algorithm 1. Implementation of classic Quicksort as given in [8] (see [2], [9] and [10] for detailed analyses).

Two pointers i and j scan the array from left and right until they hit an element that does not belong in their current subfiles. Then the elements $A[i]$ and $A[j]$ are exchanged. This “crossing pointers” technique is due to Hoare [11], [1].

QUICKSORT($A, left, right$)

```

    // Sort the array  $A$  in index range  $left, \dots, right$ . We assume a sentinel  $A[0] = -\infty$ .
1  if  $right - left \geq 1$ 
2       $p := A[right]$            // Choose rightmost element as pivot
3       $i := left - 1$ ;  $j := right$ 
4      do
5          do  $i := i + 1$  while  $A[i] < p$  end while
6          do  $j := j - 1$  while  $A[j] > p$  end while
7          if  $j > i$  then Swap  $A[i]$  and  $A[j]$  end if
8      while  $j > i$ 
9      Swap  $A[i]$  and  $A[right]$  // Move pivot to final position
10     QUICKSORT( $A, left, i - 1$ )
11     QUICKSORT( $A, i + 1, right$ )
12 end if

```

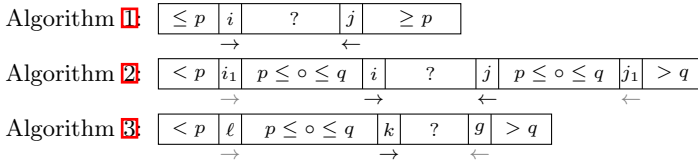


Fig. 1. Comparison of the partitioning schemes of the three Quicksort variants discussed in this paper. The pictures show the invariant maintained in partitioning.

initiated a discussion claiming his new algorithm to be superior to the runtime library’s sorting method at that time: the widely used and carefully tuned variant of classic Quicksort from [12]. Indeed, Yaroslavskiy’s Quicksort has been chosen as the new default sorting algorithm in Oracle’s Java 7 runtime library after extensive empirical performance tests.

In light of the results on multi-pivot Quicksort mentioned above, this is quite surprising and asks for explanation. Accordingly, since the new dual pivot Quicksort variant has not been analyzed in detail, yet³, corresponding average case results will be proven in this paper. Our analysis reveals the reason why dual pivot Quicksort can indeed outperform the classic algorithm and why the partitioning method of Algorithm 2 is suboptimal. It turns out that Yaroslavskiy’s partitioning method is able to take advantage of certain asymmetries in the

³ Note that the results presented in <http://iaroslavski.narod.ru/quicksort/DualPivotQuicksort.pdf> provide wrong constants and thus are insufficient for our needs.

Algorithm 2. Dual Pivot Quicksort with Sedgewick's partitioning as proposed in [2] (Program 5.1). This is an equivalent Java-like adaption of the original ALGOL-style program.

```

DUALPIVOTQUICKSORTSEDGWICK( $A, left, right$ )
    // Sort the array  $A$  in index range  $left, \dots, right$ . We assume a sentinel  $A[0] = -\infty$ .
1  if  $right - left \geq 1$ 
2       $i := left$ ;  $i_1 := left$ ;  $j := right$ ;  $j_1 := right$ ;  $p := A[left]$ ;  $q := A[right]$ 
3      if  $p > q$  then Swap  $p$  and  $q$  end if
4      while true
5           $i := i + 1$ 
6          while  $A[i] \leq q$ 
7              if  $i \geq j$  then break outer while end if // pointers have crossed
8              if  $A[i] < p$  then  $A[i_1] := A[i]$ ;  $i_1 := i_1 + 1$ ;  $A[i] := A[i_1]$  end if
9               $i := i + 1$ 
10         end while
11          $j := j - 1$ 
12         while  $A[j] \geq p$ 
13             if  $A[j] > q$  then  $A[j_1] := A[j]$ ;  $j_1 := j_1 - 1$ ;  $A[j] := A[j_1]$  end if
14             if  $i \geq j$  then break outer while end if // pointers have crossed
15              $j := j - 1$ 
16         end while
17          $A[i_1] := A[j]$ ;  $A[j_1] := A[i]$ 
18          $i_1 := i_1 + 1$ ;  $j_1 := j_1 - 1$ 
19          $A[i] := A[i_1]$ ;  $A[j] := A[j_1]$ 
20     end while
21      $A[i_1] := p$ ;  $A[j_1] := q$ 
22     DUALPIVOTQUICKSORTSEDGWICK( $A, left, i_1 - 1$ )
23     DUALPIVOTQUICKSORTSEDGWICK( $A, i_1 + 1, j_1 - 1$ )
24     DUALPIVOTQUICKSORTSEDGWICK( $A, j_1 + 1, right$ )
25 end if

```

outcomes of key comparisons. Algorithm 2 fails to utilize them, even though being based on the same abstract algorithmic idea.

2 Results

In this paper, we give the first precise average case analysis of Yaroslavskiy's dual pivot Quicksort (Algorithm 3), the new default sorting method in Oracle's Java 7 runtime library. Using these original results, we compare the algorithm to existing Quicksort variants: The classic Quicksort (Algorithm 1) and a dual pivot Quicksort as proposed by Sedgewick in [2] (Algorithm 2).

Table 1 shows formulæ for the expected number of key comparisons and swaps for all three algorithms. In terms of comparisons, the new dual pivot Quicksort by Yaroslavskiy is best. However, it needs more swaps, so whether it can outperform the classic Quicksort, depends on the relative runtime contribution of swaps and

Algorithm 3. Dual Pivot Quicksort with Yaroslavskiy's partitioning method

```

DUALPIVOTQUICKSORTYAROSLAVSKIY( $A, left, right$ )
  // Sort the array  $A$  in index range  $left, \dots, right$ . We assume a sentinel  $A[0] = -\infty$ .
  1  if  $right - left \geq 1$ 
  2     $p := A[left]$ ;  $q := A[right]$ 
  3    if  $p > q$  then Swap  $p$  and  $q$  end if
  4     $\ell := left + 1$ ;  $g := right - 1$ ;  $k := \ell$ 
  5    while  $k \leq g$ 
  6      if  $A[k] < p$ 
  7        Swap  $A[k]$  and  $A[\ell]$ 
  8         $\ell := \ell + 1$ 
  9      else
 10        if  $A[k] > q$ 
 11          while  $A[g] > q$  and  $k < g$  do  $g := g - 1$  end while
 12          Swap  $A[k]$  and  $A[g]$ 
 13           $g := g - 1$ 
 14          if  $A[k] < p$ 
 15            Swap  $A[k]$  and  $A[\ell]$ 
 16             $\ell := \ell + 1$ 
 17          end if
 18        end if
 19      end if
 20       $k := k + 1$ 
 21    end while
 22     $\ell := \ell - 1$ ;  $g := g + 1$ 
 23    Swap  $A[left]$  and  $A[\ell]$  // Bring pivots to final position
 24    Swap  $A[right]$  and  $A[g]$ 
 25    DUALPIVOTQUICKSORTYAROSLAVSKIY( $A, left, \ell - 1$ )
 26    DUALPIVOTQUICKSORTYAROSLAVSKIY( $A, \ell + 1, g - 1$ )
 27    DUALPIVOTQUICKSORTYAROSLAVSKIY( $A, g + 1, right$ )
 28  end if

```

Table 1. Exact expected number of comparisons and swaps of the three Quicksort variants in the random permutation model. The results for Algorithm [1](#) are taken from [\[10\]](#) p. 334] (for $M = 1$). $\mathcal{H}_n = \sum_{i=1}^n \frac{1}{i}$ is the n th harmonic number, which is asymptotically $\mathcal{H}_n = \ln n + 0.577216 \dots + \mathcal{O}(n^{-1})$ as $n \rightarrow \infty$.

	Comparisons	Swaps
Classic Quicksort (Algorithm 1)	$2(n+1)\mathcal{H}_{n+1} - \frac{8}{3}(n+1)$ $\approx 2n \ln n - 1.51n + \mathcal{O}(\ln n)$	$\frac{1}{3}(n+1)\mathcal{H}_{n+1} - \frac{7}{9}(n+1) + \frac{1}{2}$ $\approx 0.33n \ln n - 0.58n + \mathcal{O}(\ln n)$
Sedgewick (Algorithm 2)	$\frac{32}{15}(n+1)\mathcal{H}_{n+1} - \frac{856}{225}(n+1) + \frac{3}{2}$ $\approx 2.13n \ln n - 2.57n + \mathcal{O}(\ln n)$	$\frac{4}{5}(n+1)\mathcal{H}_{n+1} - \frac{19}{25}(n+1) - \frac{1}{4}$ $\approx 0.8n \ln n - 0.30n + \mathcal{O}(\ln n)$
Yaroslavskiy (Algorithm 3)	$\frac{19}{10}(n+1)\mathcal{H}_{n+1} - \frac{711}{200}(n+1) + \frac{3}{2}$ $\approx 1.9n \ln n - 2.46n + \mathcal{O}(\ln n)$	$\frac{3}{5}(n+1)\mathcal{H}_{n+1} - \frac{27}{100}(n+1) - \frac{7}{12}$ $\approx 0.6n \ln n + 0.08n + \mathcal{O}(\ln n)$

comparisons, which in turn differ from machine to machine. Section 4 shows some running times, where indeed Algorithm 3 was fastest.

Remarkably, the new algorithm is significantly better than Sedgwick's dual pivot Quicksort in both measures. Given that Algorithms 2 and 3 are based on the same algorithmic idea, the considerable difference in costs is surprising. The explanation of the superiority of Yaroslavskiy's variant is a major discovery of this paper. Hence, we first give a qualitative teaser of it. Afterwards, Section 3 gives a thorough analysis, making the arguments precise.

2.1 The Superiority of Yaroslavskiy's Partitioning Method

Let $p < q$ be the two pivots. For partitioning, we need to determine for every $x \notin \{p, q\}$ whether $x < p$, $p < x < q$ or $q < x$ holds by comparing x to p and/or q . Assume, we first compare x to p , then averaging over all possible values for p , q and x , there is a $1/3$ chance that $x < p$ – in which case we are done. Otherwise, we still need to compare x and q . The expected number of comparisons for one element is therefore $1/3 \cdot 1 + 2/3 \cdot 2 = 5/3$. For a partitioning step with n elements including pivots p and q , this amounts to $5/3 \cdot (n - 2)$ comparisons in expectation.

In the random permutation model, knowledge about an element $y \neq x$ does not tell us whether $x < p$, $p < x < q$ or $q < x$ holds. Hence, one could think that any partitioning method should need at least $5/3 \cdot (n - 2)$ comparisons in expectation. But this is not the case.

The reason is the independence assumption above, which only holds true for algorithms that do comparisons at exactly *one location in the code*. But Algorithms 2 and 3 have several compare-instructions at different locations, and how often those are reached *depends* on the pivots p and q . Now of course, the number of elements smaller, between and larger p and q , directly depends on p and q , as well! So if a comparison is executed often if p is large, it is clever to first check $x < p$ there: The comparison is done more often than on average if and only if the probability for $x < p$ is larger than on average. Therefore, the expected number of comparisons can drop below the “lower bound” $5/3$ for this element!

And this is exactly, where Algorithms 2 and 3 differ: Yaroslavskiy's partitioning always evaluates the “better” comparison first, whereas in Sedgwick's dual pivot Quicksort this is not the case. In Section 3.3, we will give this a more quantitative meaning based on our analysis.

3 Average Case Analysis of Dual Pivot Quicksort

We assume input sequences to be random permutations, i. e. each permutation π of elements $\{1, \dots, n\}$ occurs with probability $1/n!$. The first and last elements are chosen as pivots; let the smaller one be p , the larger one q .

Note that all Quicksort variants in this paper fulfill the following property:

Property 1. Every key comparison involves a pivot element of the current partitioning step.

3.1 Solution to the Dual Pivot Quicksort Recurrence

In [13], Hennequin shows that Property 1 is a sufficient criterion for *preserving randomness* in subfiles, i.e. if the whole array is a (uniformly chosen) random permutation of its elements, so are the subproblems Quicksort is recursively invoked on. This allows us to set up a recurrence relation for the expected costs, as it ensures that all partitioning steps of a subarray of size k have the same expected costs as the initial partitioning step for a random permutation of size k .

The expected costs C_n for sorting a random permutation of length n by any dual pivot Quicksort with Property 1 satisfy the following recurrence relation:

$$\begin{aligned} C_n &= \sum_{1 \leq p < q \leq n} \Pr[\text{pivots } (p, q)] \cdot (\text{partitioning costs} + \text{recursive costs}) \\ &= \sum_{1 \leq p < q \leq n} \frac{2}{n(n-1)} (\text{partitioning costs} + C_{p-1} + C_{q-p-1} + C_{n-q}), \end{aligned}$$

for $n \geq 3$ with base cases $C_0 = C_1 = 0$ and $C_2 = d$ ⁴

We confine ourselves to linear expected partitioning costs $a(n+1) + b$, where a and b are constants depending on the kind of costs we analyze. The recurrence relation can then be solved by standard techniques – the detailed calculations can be found in Appendix A. The closed form for C_n is

$$C_n = \frac{6}{5}a \cdot (n+1) \left(\mathcal{H}_{n+1} - \frac{1}{5} \right) + \left(-\frac{3}{2}a + \frac{3}{10}b + \frac{1}{10}d \right) \cdot (n+1) - \frac{1}{2}b,$$

which is valid for $n \geq 4$ with $\mathcal{H}_n = \sum_{i=1}^n \frac{1}{i}$ the n th harmonic number.

3.2 Costs of One Partitioning Step

In this section, we analyze the expected number of swaps and comparisons used in the first partitioning step on a random permutation of $\{1, \dots, n\}$. The results are summarized in Table 2. To state the proofs, we need to introduce some notation.

Table 2. Expected costs of the first partitioning step for the two dual pivot Quicksort variants on a random permutation of length n (for $n \geq 3$)

	Comparisons	Swaps
Sedgewick (Algorithm 2)	$\frac{16}{9}(n+1) - 3 - \frac{2}{3} \frac{1}{n(n-1)}$	$\frac{2}{3}(n+1) + \frac{1}{2}$
Yaroslavskiy (Algorithm 3)	$\frac{19}{12}(n+1) - 3$	$\frac{1}{2}(n+1) + \frac{7}{6}$

⁴ d can easily be determined manually: For Algorithm 3 it is 1 for comparisons and $\frac{5}{2}$ for swaps and for Algorithm 2 we have $d = 2$ for comparisons and $d = \frac{5}{2}$ for swaps.

Notation. Let S be the set of all elements smaller than both pivots, M those in the middle and L the large ones, i. e.

$$S := \{1, \dots, p-1\}, \quad M := \{p+1, \dots, q-1\}, \quad L := \{q+1, \dots, n\}.$$

Then, by Property [1](#) the algorithm cannot distinguish $x \in C$ from $y \in C$ for any $C \in \{S, M, L\}$. Hence, for analyzing partitioning costs, we replace all non-pivot elements by s , m or l when they are elements of S , M or L , respectively. Obviously, all possible results of a partitioning step correspond to the same word $s \dots s p m \dots m q l \dots l$. The following example will demonstrate these definitions.

Example 1. Example permutation before ... and after partitioning.

$$\begin{array}{cccccccccc} \overset{p}{\boxed{2}} & 4 & 7 & 8 & 1 & 6 & 9 & 3 & \overset{q}{\boxed{5}} & \\ p & m & l & l & s & l & l & m & q & \end{array} \qquad \begin{array}{cccccccccc} 1 & \boxed{2} & 4 & 3 & \boxed{5} & 6 & 9 & 8 & 7 & \\ s & p & m & m & q & l & l & l & l & \end{array}$$

Next, we define position sets $\mathcal{S}$, $\mathcal{M}$ and $\mathcal{L}$ as follows:

$$\begin{array}{ll} \mathcal{S} := \{2, \dots, p\}, & \text{in the example:} \\ \mathcal{M} := \{p+1, \dots, q-1\}, & \begin{array}{cccccccccc} \mathcal{S} & \mathcal{M} & \mathcal{M} & \mathcal{L} & \mathcal{L} & \mathcal{L} & \mathcal{L} & & & \\ \boxed{2} & 4 & 7 & 8 & 1 & 6 & 9 & 3 & \boxed{5} & \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & \end{array} \\ \mathcal{L} := \{q, \dots, n-1\}. & \end{array}$$

Now, we can formulate the main quantities occurring in the analysis below: For a given permutation, $c \in \{s, m, l\}$ and a set of positions $\mathcal{P} \subset \{1, \dots, n\}$, we write $c @ \mathcal{P}$ for the number of c -type elements occurring *at positions* in $\mathcal{P}$ of the permutation. In our last example, $\mathcal{M} = \{3, 4\}$ holds. At these positions, we find elements 7 and 8 (before partitioning), both belonging to L . Thus, $l @ \mathcal{M} = 2$, whereas $s @ \mathcal{M} = m @ \mathcal{M} = 0$.

Now consider a *random* permutation. Then $c @ \mathcal{P}$ becomes a random variable. In the analysis, we will encounter the conditional expectation of $c @ \mathcal{P}$ *given* that the random permutation induces the pivots p and q , i. e. the first and last element of the permutation are p and q or q and p , respectively. We abbreviate this quantity as $\mathbb{E}[c @ \mathcal{P} | p, q]$. As the number $\#c$ of c -type elements only depends on the pivots, not on the permutation itself, $\#c$ is a fully determined constant in $\mathbb{E}[c @ \mathcal{P} | p, q]$. Hence, given pivots p and q , $c @ \mathcal{P}$ is a hypergeometrically distributed random variable: For the c -type elements, we draw their $\#c$ positions out of $n-2$ possible positions via sampling without replacement. Drawing a position in $\mathcal{P}$ is a ‘success’, a position not in $\mathcal{P}$ is a ‘failure’.

Accordingly, $\mathbb{E}[c @ \mathcal{P} | p, q]$ can be expressed as the mean of this hypergeometric distribution: $\mathbb{E}[c @ \mathcal{P} | p, q] = \#c \cdot \frac{|\mathcal{P}|}{n-2}$. By the law of total expectation, we finally have

$$\begin{aligned} \mathbb{E}[c @ \mathcal{P}] &= \sum_{1 \leq p < q \leq n} \mathbb{E}[c @ \mathcal{P} | p, q] \cdot \Pr[\text{pivots } (p, q)] \\ &= \frac{2}{n(n-1)} \sum_{1 \leq p < q \leq n} \#c \cdot \frac{|\mathcal{P}|}{n-2}. \end{aligned}$$

Comparisons in Algorithm 3. Algorithm 3 contains five places where key comparisons are used, namely in lines 3, 6, 10, 11 and 14. Line 3 compares the two pivots and is executed exactly once. Line 6 is executed once per value for k except for the last increment, where we leave the loop before the comparison is done. Similarly, line 11 is run once for every value of g except for the last one.

The comparison in line 10 can only be reached, when line 6 made the ‘else’-branch apply. Hence, line 10 causes as many comparisons as k attains values with $A[k] \geq p$. Similarly, line 14 is executed once for all values of g where $A[g] \leq q$.⁵

At the end, q gets swapped to position g (line 24). Hence we must have $g = q$ there. Accordingly, g attains values $\mathcal{G} = \{n-1, n-2, \dots, q\} = \mathcal{L}$ at line 11. We always leave the outer while loop with $k = g+1$ or $k = g+2$. In both cases, k (at least) attains values $\mathcal{K} = \{2, \dots, q-1\} = \mathcal{S} \cup \mathcal{M}$ in line 11. The case “ $k = g+2$ ” introduces an additional term of $3 \cdot \frac{n-q}{n-2}$; see Appendix B for the detailed discussion.

Summing up all contributions yields the conditional expectation $c_n^{p,q}$ of the number of comparisons needed in the first partitioning step for a random permutation, given it implies pivots p and q :

$$\begin{aligned}
 c_n^{p,q} &= 1 + |\mathcal{K}| + |\mathcal{G}| + (\mathbb{E}[m @ \mathcal{K} | p, q] + \mathbb{E}[l @ \mathcal{K} | p, q]) \\
 &\quad + (\mathbb{E}[s @ \mathcal{G} | p, q] + \mathbb{E}[m @ \mathcal{G} | p, q]) \\
 &\quad + 3 \cdot \frac{n-q}{n-2} \\
 &= n-1 + ((q-p-1) + (n-q)) \frac{q-2}{n-2} \\
 &\quad + ((p-1) + (q-p-1)) \frac{n-q}{n-2} \\
 &\quad + 3 \cdot \frac{n-q}{n-2} \\
 &= n-1 + (n-p-1) \frac{q-2}{n-2} + (q+1) \frac{n-q}{n-2}.
 \end{aligned}$$

Now, by the law of total expectation, the expected number of comparisons in the first partitioning step for a random permutation of $\{1, \dots, n\}$ is

$$\begin{aligned}
 c_n &:= \mathbb{E} c_n^{p,q} = \frac{2}{n(n-1)} \sum_{p=1}^{n-1} \sum_{q=p+1}^n c_n^{p,q} \\
 &= n-1 + \frac{2}{n(n-1)(n-2)} \sum_{p=1}^{n-1} (n-p-1) \sum_{q=p+1}^n (q-2) \\
 &\quad + \frac{2}{n(n-1)(n-2)} \sum_{q=2}^n (n-q)(q+1) \sum_{p=1}^{q-1} 1 \\
 &= n-1 + \left(\frac{5}{12}(n+1) - \frac{4}{3}\right) + \frac{1}{6}(n+3) = \frac{19}{12}(n+1) - 3.
 \end{aligned}$$

⁵ Line 12 just swapped $A[k]$ and $A[g]$. So even though line 14 literally says “ $A[k] < p$ ”, this comparison actually refers to an element first reached as $A[g]$.

Swaps in Algorithm 3. Swaps happen in Algorithm 3 in lines 3, 7, 12, 15, 23 and 24. Lines 23 and 24 are both executed exactly once. Line 3 once swaps the pivots if needed, which happens with probability $1/2$. For each value of k with $A[k] < p$, one swap occurs in line 7. Line 12 is executed for every value of k having $A[k] > q$. Finally, line 15 is reached for all values of g where $A[g] < p$ (see footnote 5).

Using the ranges $\mathcal{K}$ and $\mathcal{G}$ from above, we obtain $s_n^{p,q}$, the conditional expected number of swaps for partitioning a random permutation, given pivots p and q . There is an additional contribution of $\frac{n-q}{n-2}$ when k stops with $k = g + 2$ instead of $k = g + 1$. As for comparisons, its detailed discussion is deferred to Appendix B.

$$\begin{aligned} s_n^{p,q} &= \frac{1}{2} + 1 + 1 + \mathbb{E}[s @ \mathcal{K} | p, q] + \mathbb{E}[l @ \mathcal{K} | p, q] + \mathbb{E}[s @ \mathcal{G} | p, q] + \frac{n-q}{n-2} \\ &= \frac{5}{2} + (p-1)\frac{q-2}{n-2} + (n-q)\frac{q-2}{n-2} + (p-1)\frac{n-q}{n-2} + \frac{n-q}{n-2} \\ &= \frac{5}{2} + (n+p-q-1)\frac{q-2}{n-2} + p \cdot \frac{n-q}{n-2}. \end{aligned}$$

Averaging over all possible p and q again, we find

$$\begin{aligned} s_n &:= \mathbb{E} s_n^{p,q} = \frac{5}{2} + \frac{2}{n(n-1)(n-2)} \sum_{q=2}^n (q-2) \sum_{p=1}^{q-1} (n+p-q-1) \\ &\quad + \frac{2}{n(n-1)(n-2)} \sum_{q=2}^n (n-q) \sum_{p=1}^{q-1} p \\ &= \frac{5}{2} + \left(\frac{5}{12}(n+1) - \frac{4}{3}\right) + \frac{1}{12}(n+1) = \frac{1}{2}(n+1) + \frac{7}{6}. \end{aligned}$$

Comparisons in Algorithm 2. Key comparisons happen in Algorithm 2 in lines 3, 6, 8, 12 and 13. Lines 6 and 12 are executed once for every value of i respectively j (without the initialization values *left* and *right* respectively). Line 8 is reached for all values of i with $A[i] \leq q$ except for the last value. Finally, the comparison in line 13 gets executed for every value of j having $A[j] \geq p$.

The value-ranges of i and j are $\mathcal{I} = \{2, \dots, \hat{i}\}$ and $\mathcal{J} = \{n-1, n-2, \dots, \hat{j}\}$ respectively, where $\hat{i}$ depends on the positions of m -type elements. So, lines 6 and 12 together contribute $|\mathcal{I}| + |\mathcal{J}| = n-1$ comparisons. For lines 8 and 13, we get additionally

$$(\mathbb{E}[s @ \mathcal{I}' | p, q] + \mathbb{E}[m @ \mathcal{I}' | p, q]) + (\mathbb{E}[m @ \mathcal{J} | p, q] + \mathbb{E}[l @ \mathcal{J} | p, q])$$

many comparisons (in expectation), where $\mathcal{I}' := \mathcal{I} \setminus \hat{i}$. As i and j cannot meet on an m -type element (both would not stop), $m @ \{\hat{i}\} = 0$, so

$$\mathbb{E}[m @ \mathcal{I}' | p, q] + \mathbb{E}[m @ \mathcal{J} | p, q] = q - p - 1.$$

Positions of m -type elements do not contribute to $s @ \mathcal{I}'$ (and $l @ \mathcal{J}$) by definition. Hence, it suffices to determine the number of non- m -elements located

Table 3. $\mathbb{E}[c @ \mathcal{P}]$ for $c = s, m, l$ and $P = \mathcal{S}, \mathcal{M}, \mathcal{L}$

	$\mathcal{S}$	$\mathcal{M}$	$\mathcal{L}$
s	$\frac{1}{6}(n-1)$	$\frac{1}{12}(n-3)$	$\frac{1}{12}(n-3)$
m	$\frac{1}{12}(n-3)$	$\frac{1}{6}(n-1)$	$\frac{1}{12}(n-3)$
l	$\frac{1}{12}(n-3)$	$\frac{1}{12}(n-3)$	$\frac{1}{6}(n-1)$

at positions in $\mathcal{I}'$. A glance at Figure 1 suggests to count non- m -type elements left of (and including) the last value of i_1 , which is p . So, the first $p-1$ of all $(p-1) + (n-q)$ non- m -positions are contained in $\mathcal{I}'$, thus $\mathbb{E}[s @ \mathcal{I}' | p, q] = (p-1) \frac{p-1}{(p-1)+(n-q)}$. Similarly, we can show that $l @ \mathcal{J}$ is the number of l -type elements right of i_1 's largest value: $\mathbb{E}[l @ \mathcal{J} | p, q] = (n-q) \frac{n-q}{(p-1)+(n-q)}$. Summing up all contributions, we get

$$c_n'^{p,q} = n-1 + q - p - 1 + (p-1) \frac{p-1}{(p-1)+(n-q)} + (n-q) \frac{n-q}{(p-1)+(n-q)}.$$

Taking the expectation over all possible pivot values yields

$$c_n' = \frac{2}{n(n-1)} \sum_{p=1}^{n-1} \sum_{q=p+1}^n c_n'^{p,q} = \frac{16}{9}(n+1) - 3 - \frac{2}{3} \frac{1}{n(n-1)}.$$

This is not a linear function and hence does not directly fit our solution of the recurrence from Section 3.1. The exact result given in Table 1 is easily proven by induction. Dropping summand $-\frac{2}{3} \frac{1}{n(n-1)}$ and inserting the linear part into the recurrence relation, still gives the correct leading term; in fact, the error is only $\frac{1}{90}(n+1)$.

Swaps in Algorithm 2. The expected number of swaps has already been analyzed in [2]. There, it is shown that Sedgewick's partitioning step needs $\frac{2}{3}(n+1)$ swaps, on average – excluding the pivot swap in line 3. As we count this swap for Algorithm 3, we add $\frac{1}{2}$ to the expected value for Algorithm 2, for consistency.

3.3 Superiority of Yaroslavskiy's Partitioning Method – Continued

In this section, we abbreviate $\mathbb{E}[c @ \mathcal{P}]$ by $E_c^{\mathcal{P}}$ for conciseness. It is quite enlightening to compute $E_c^{\mathcal{P}}$ for $c = s, m, l$ and $\mathcal{P} = \mathcal{S}, \mathcal{M}, \mathcal{L}$, see Table 3. There is a remarkable *asymmetry*, e.g. averaging over all permutations, *more than half* of all l -type elements are located at positions in $\mathcal{L}$. Thus, if we *know* we are looking at a position in $\mathcal{L}$, it is much more advantageous to first compare with q , as with probability $> \frac{1}{2}$, the element is $> q$. This results in an expected number of comparisons $< \frac{1}{2} \cdot 2 + \frac{1}{2} \cdot 1 = \frac{3}{2} < \frac{5}{3}$. Line 11 of Algorithm 3 is exactly of this type. Hence, Yaroslavskiy's partitioning method exploits the knowledge about the different position sets comparisons are reached for. Conversely, lines 6 and 12 in Algorithm 2 are of the opposite type: They check the unlikely outcome first.

We can roughly approximate the expected number of comparisons in Algorithms [2](#) and [3](#) by expressing them in terms of the quantities from Table [3](#) (using $\mathcal{K} = \mathcal{S} \cup \mathcal{M}$, $\mathcal{G} \approx \mathcal{L}$ and $E_s^{\mathcal{I}'} + E_l^{\mathcal{J}} \approx E_s^{\mathcal{S}} + E_l^{\mathcal{L}} + E_s^{\mathcal{M}}$):

$$\begin{aligned} c'_n &= n - 1 + \mathbb{E} \#m + E_s^{\mathcal{I}'} + E_l^{\mathcal{J}} \\ &\approx n + (E_m^{\mathcal{S}} + E_m^{\mathcal{M}} + E_m^{\mathcal{L}}) + (E_s^{\mathcal{S}} + E_l^{\mathcal{L}} + E_s^{\mathcal{M}}) \\ &\approx (1 + 3 \cdot \tfrac{1}{12} + 3 \cdot \tfrac{1}{6})n \approx 1.75n \quad (\text{exact: } 1.78n - 1.22 + o(1)) \\ c_n &= n + E_m^{\mathcal{K}} + E_l^{\mathcal{K}} + E_s^{\mathcal{G}} + E_m^{\mathcal{G}} \\ &\approx n + (E_m^{\mathcal{S}} + E_m^{\mathcal{M}}) + (E_l^{\mathcal{S}} + E_l^{\mathcal{M}}) + E_s^{\mathcal{L}} + E_m^{\mathcal{L}} \\ &\approx (1 + 5 \cdot \tfrac{1}{12} + 1 \cdot \tfrac{1}{6})n \approx 1.58n \quad (\text{exact: } 1.58n - 0.75) \end{aligned}$$

Note that both terms involve six ‘ $E_c^{\mathcal{P}}$ -terms’, but Algorithm [2](#) has *three* ‘expensive’ terms, whereas Algorithm [3](#) only has *one* such term.

4 Some Running Times

Extensive performance tests have already been done for Yaroslavskiy’s dual pivot Quicksort. However, those were based on an optimized implementation intended for production use. In Figure [2](#) we provide some running times of the basic variants as given in Algorithms [1](#), [2](#) and [3](#) to directly evaluate the algorithmic ideas, complementing our analysis.

Note: This is not intended to replace a thorough performance study, but merely to demonstrate that Yaroslavskiy’s partitioning method performs well – at least on our machine.

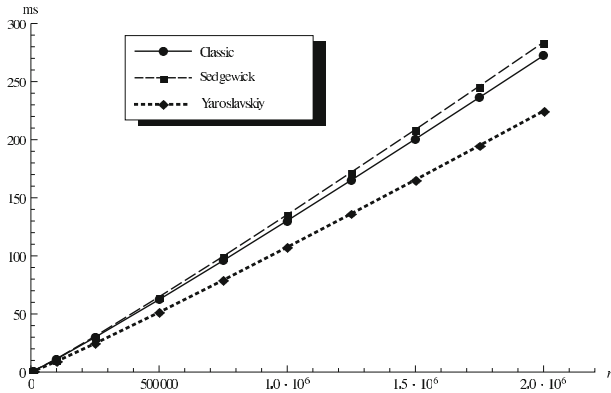


Fig. 2. Running times of Java implementations of Algorithms [1](#), [2](#) and [3](#) on an Intel Core 2 Duo P8700 laptop. The plot shows the average running time of 1000 random permutations of each size.

5 Conclusion and Future Work

Having understood how the new Quicksort saves key comparisons, there are plenty of future research directions. The question if and how the new Quicksort can compensate for the many extra swaps it needs, calls for further examination. One might conjecture that comparisons have a higher runtime impact than swaps. It would be interesting to see a closer investigation – empirically or theoretically.

In this paper, we only considered the most basic implementation of dual pivot Quicksort. Many suggestions to improve the classic algorithm are also applicable to it. We are currently working on the effect of selecting the pivot from a larger sample and are keen to see the performance impacts.

Being intended as a standard sorting method, it is not sufficient for the new Quicksort to perform well on random permutations. One also has to take into account other input distributions, most notably the occurrence of equal keys or biases in the data. This might be done using Maximum Likelihood Analysis as introduced in [14], which also helped us much in discovering the results of this paper. Moreover, Yaroslavskiy's partitioning method can be used to improve Quickselect. Our corresponding results are omitted due to space constraints.

References

1. Hoare, C.A.R.: Quicksort. *The Computer Journal* 5(1), 10–16 (1962)
2. Sedgewick, R.: Quicksort. Phd thesis, Stanford University (1975)
3. Hennequin, P.: Analyse en moyenne d'algorithme, tri rapide et arbres de recherche. Ph.d. thesis, Ecole Polytechnique, Palaiseau (1991)
4. Frazer, W.D., McKellar, A.C.: Samplesort: A Sampling Approach to Minimal Storage Tree Sorting. *Journal of the ACM* 17(3), 496–507 (1970)
5. Sanders, P., Winkel, S.: Super Scalar Sample Sort. In: Albers, S., Radzik, T. (eds.) *ESA 2004*. LNCS, vol. 3221, pp. 784–796. Springer, Heidelberg (2004)
6. Leischner, N., Osipov, V., Sanders, P.: GPU sample sort. In: 2010 IEEE International Symposium on Parallel Distributed Processing IPDPS, pp. 1–10. IEEE (2009)
7. Blleloch, G.E., Leiserson, C.E., Maggs, B.M., Plaxton, C.G., Smith, S.J., Zagha, M.: A comparison of sorting algorithms for the connection machine CM-2. In: *Proceedings of the Third Annual ACM Symposium on Parallel Algorithms and Architectures - SPAA 1991*, pp. 3–16. ACM Press, New York (1991)
8. Sedgewick, R.: Implementing Quicksort programs. *Comm. ACM* 21(10), 847–857 (1978)
9. Sedgewick, R.: Quicksort with Equal Keys. *SIAM Journal on Computing* 6(2), 240–267 (1977)
10. Sedgewick, R.: The analysis of Quicksort programs. *Acta Inf.* 7(4), 327–355 (1977)
11. Hoare, C.A.R.: Algorithm 63: Partition. *Comm.* 4(7), 321 (1961)
12. Bentley, J.L.J., McIlroy, M.D.: Engineering a sort function. *Software: Practice and Experience* 23(11), 1249–1265 (1993)
13. Hennequin, P.: Combinatorial analysis of Quicksort algorithm. *Informatique Théorique et Applications* 23(3), 317–333 (1989)
14. Laube, U., Nebel, M.E.: Maximum likelihood analysis of algorithms and data structures. *Theoretical Computer Science* 411(1), 188–212 (2010)